

Movies Search Engine

Data Mining Project Report

Phase 1 + Phase 2 + Bonus Implementation

10,042

Documents

9,076

Movies

966

TV Shows

Prepared by:

Malak Osama - 202402421

Rowida Sayed - 202402291

An intelligent retrieval system for movies and TV records using preprocessing, inverted indexing, TF-IDF ranking, query expansion, clustering, recommendations, and a polished web interface.

Executive Summary

This report documents the complete Movies Search Engine project, starting from data acquisition and preprocessing in Phase 1 and ending with an advanced searchable web application in Phase 2. The final system uses a cleaned dataset of 10,042 movie and TV records, builds searchable text from titles and abstracts, ranks results using TF-IDF and cosine similarity, expands user queries with related terms and relevance feedback, and presents results through a professional Flask dashboard.

Project direction

The project is designed as a practical data mining application: the raw data was collected and cleaned first, then transformed into a searchable structure, then used for search, recommendation, clustering, analytics, and interactive visualization.

Table of Contents

- 1. Project Overview
- 2. Dataset Description
- 3. Phase 1: Data Collection and Preprocessing
- 4. Indexing Design
- 5. Phase 2: Query Processing and Search Engine
- 6. Query Expansion and Ranking
- 7. Bonus Web Application
- 8. Evaluation and Results
- 9. Insights, Limitations, and Future Work
- 10. Conclusion

1. Project Overview

The goal of the project is to build a complete search engine for a movies dataset. The system does not only display records; it processes text, builds search structures, ranks documents, expands queries, and presents analytical views. This makes the project closer to a real information retrieval and data mining workflow.

10,042	4.25	1897-2026	TF-IDF
Total Records	Average Rating	Year Range	Search Mode

Main Objectives

- Collect movie and TV data from multiple sources and consolidate it into one usable dataset.
- Clean and preprocess text fields so that titles and abstracts can be used reliably for search.
- Build an inverted index and TF-IDF representation to support efficient document retrieval.
- Implement query processing that applies the same cleaning logic used during indexing.
- Improve search quality using query expansion and relevance feedback.
- Create a user-friendly web interface with filters, analytics, recommendations, clustering, and network summary.

2. Dataset Description

The submitted system uses the preprocessed dataset generated from Phase 1. Each row represents a document in the search engine. A document may be a movie or a TV record, and every document is assigned a unique doc_id.

Column	Purpose
doc_id	Unique identifier used during indexing and retrieval.
title	Displayed title and part of the searchable text.
type	Content type, mainly movie or TV. Used for filtering and analytics.
year	Release year used in filters, charts, and clustering.
rating	Numerical rating used in insights, ranking boost, and clustering.
original_abstract	Readable description shown in the search results.
processed_text	Cleaned and stemmed version used by the search engine.

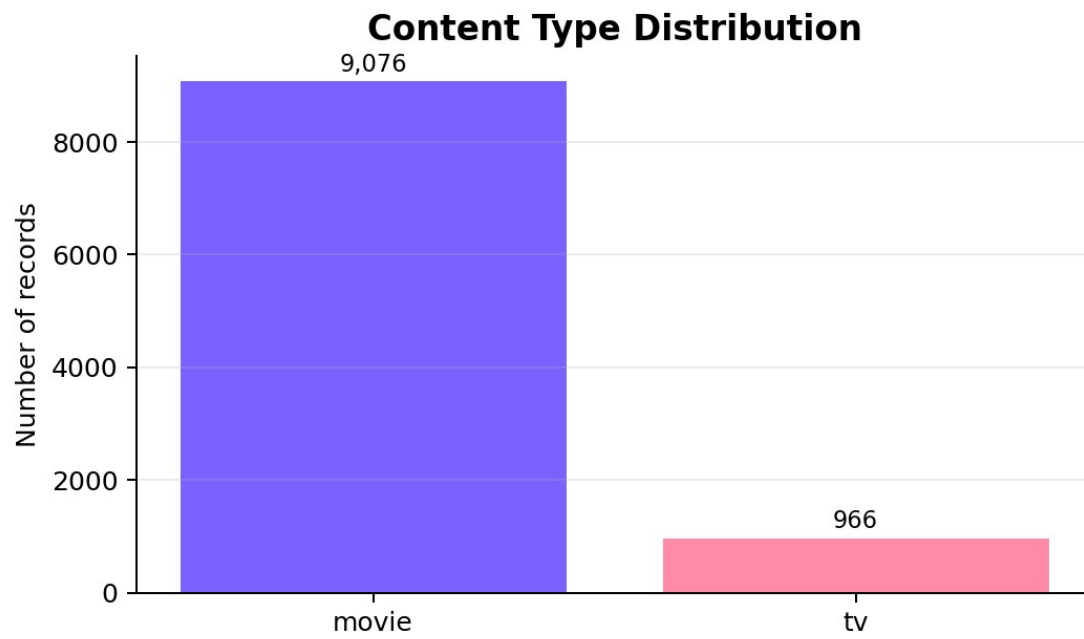


Figure 1. Distribution of movies and TV records in the final dataset.

3. Phase 1: Data Collection and Preprocessing

3.1 Data Collection

Phase 1 focused on collecting enough real movie-related data to support retrieval and analysis. The project combines structured sources and API-based data. TMDB and OMDb were used for movie and TV metadata, while the Kaggle movies file provided an additional structured source. The collected data was merged into a unified format before preprocessing.

Source	Role in the project	Used fields
TMDB API	Collected Egyptian movie and TV metadata across many pages.	title, type, year, rating, votes, popularity, abstract
OMDb API	Supplemented selected records with additional movie information.	title, year, type, plot/abstract fields
Kaggle movies dataset	Provided a larger movie catalogue for variety and comparison.	title, overview, popularity, release date, ratings

3.2 Preprocessing Pipeline

The preprocessing stage converts raw descriptions into consistent searchable text. This step was already completed and graded in Phase 1, so the final application uses the cleaned file directly while still documenting the process clearly.

- Removed duplicate records and empty abstracts.
- Normalized text by converting it to lowercase.
- Removed punctuation and noisy characters.
- Tokenized each title and abstract into words.
- Removed stopwords to keep meaningful terms only.
- Applied stemming to reduce related word forms to a shared root.
- Created `processed_text` by combining cleaned title and abstract content.

Preprocessed file used in the final system

The application loads `movies_preprocessed.csv`. This is appropriate because preprocessing was handled in Phase 1, and Phase 2 focuses on search, ranking, expansion, interface, analytics, and mining features.

4. Indexing Design

Indexing transforms documents into structures that can be searched quickly. The project includes an inverted index concept in the notebook phase and a TF-IDF matrix in the final Flask application. The inverted index maps each term to the documents where it appears, while the TF-IDF matrix supports ranking by term importance.

Component	Explanation
Documents dictionary	Maps <code>doc_id</code> to <code>processed_text</code> .
Inverted index	Stores each term with document IDs and term frequency.
TF-IDF vectorizer	Creates weighted vectors for titles and abstracts using unigrams and bigrams.
Cosine similarity	Compares query vector with document vectors to rank

	results.
Title boost	Adds a small boost when query terms appear directly in the title.
Rating boost	Adds a controlled rating-based boost without dominating textual relevance.



End-to-end workflow used in the submitted Movies Search Engine project

Figure 2. Full workflow from raw sources to final dashboard.

5. Phase 2: Query Processing and Search Engine

5.1 Query Processing

When a user searches, the query goes through the same general cleaning logic used in preprocessing. This keeps the query representation compatible with the indexed movie documents.

- Convert the query to lowercase.
- Remove punctuation and unsupported characters.
- Remove common stopwords such as the, a, and, of, and movie.
- Apply a lightweight stemming function.
- Use the cleaned terms for expansion and ranking.

5.2 Retrieval and Ranking

The final query is represented as a TF-IDF vector. The system then calculates cosine similarity between the query and every document vector. Results are filtered by user constraints, sorted by final score, and returned with rank, title, type, year, rating, score, and abstract.

Ranking Signal	Why it improves results
TF-IDF cosine similarity	Ranks documents according to matching important terms.
Query expansion	Adds related terms so the system can find semantically close records.
Title boost	Helps exact title or title-related matches appear higher.
Rating boost	Slightly favors well-rated records while keeping text relevance primary.
Filters	Allows users to narrow results by type, year range, and minimum rating.

6. Query Expansion and Recommendations

The system uses two forms of query expansion. First, a manually designed synonym map adds terms for common search themes such as crime, love, war, family, horror, comedy, Egypt, action, and space. Second, relevance feedback analyzes top-ranked documents from the initial query and adds high-weight terms from those results.

Example

A query such as crime drug is expanded with related terms like criminal, police, murder, detective, gang, and syndicate. This makes the search more flexible than strict keyword matching.

Recommendations are generated by finding a title match, calculating similarity between that movie vector and all other movie vectors, and returning the most similar records. This adds a data mining feature beyond basic search.

7. Bonus Web Application

For the bonus component, the project includes a complete Flask web application. It is designed to look and feel like a real mini search product rather than a notebook-only submission. The dashboard gives the user search controls, structured results, analytics, clustering summaries, recommendations, and network-style statistics.

Bonus Feature	Implementation detail
Professional UI	HTML, CSS, and JavaScript dashboard served by Flask.
Search panel	Query input, content type filter, year filters, minimum rating, and expansion toggle.
Analytics endpoint	Returns type counts, rating bins, yearly counts, top-rated items, and clusters.
Recommendations endpoint	Returns similar movies based on TF-IDF similarity.
Network summary endpoint	Summarizes document nodes, type nodes, important text terms, and estimated edges.
Easy running script	run_backend.bat starts the backend quickly on Windows.

8. Evaluation and Results

The system was tested using varied queries covering different topics. Each query returns ranked results and shows the processed query and expanded terms. The response time is fast because TF-IDF vectors are built once when the application starts.

Query	Processed Query	Sample Expanded Terms	Returned	Time	Top Result
crime drug	crime drug	crime, drug, criminal, police, murder	5	0.0167s	Novocaine
love story	love story	love, story, romance, romantic, relationship	5	0.0158s	Love Story
space adventure	space adventure	space, adventure, planet, galaxy, astronaut	5	0.0156s	Space Pirate Captain Harlock
war soldier	war soldier	war, soldier, battle, army, military	5	0.0157s	Saints and Soldiers
family drama	family drama	family, drama, mother, father, son	5	0.0159s	Paa
magic school	magic school	magic, school, wizard, witch, spell	5	0.0155s	Selah Al Talamiz
egypt comedy	egypt comedy	egypt, comedy, egyptian, cairo, alexandria	5	0.0157s	Arab London
police mystery	police mystery	police, mystery, box, polic, ismail	5	0.0156s	Mystery Box

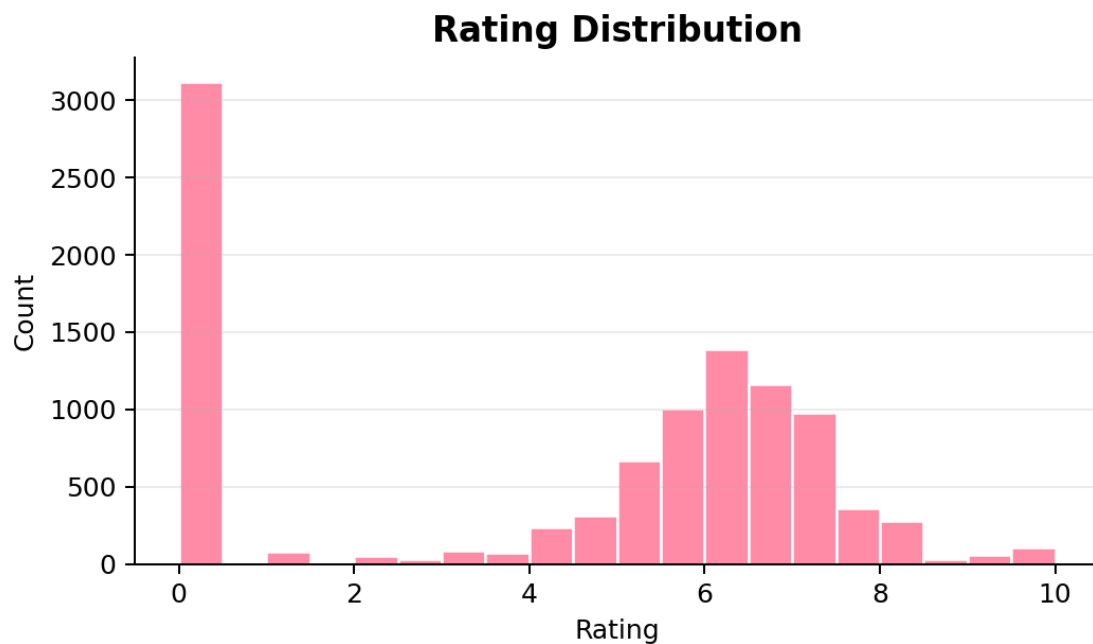


Figure 3. Rating distribution after preprocessing.

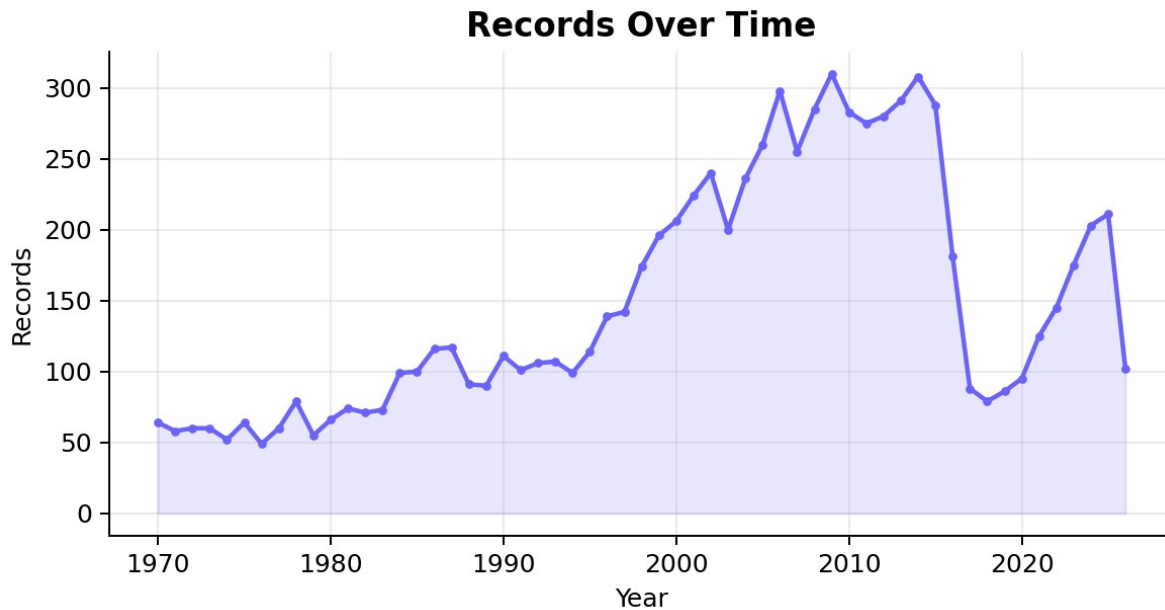


Figure 4. Number of available records over time.

9. Insights, Limitations, and Future Work

Insights

- The dataset contains 10,042 usable records, which is enough to demonstrate retrieval, ranking, and mining features.
- Movies form the largest portion of the dataset, with 9,076 movie records compared with 966 TV records.
- The average rating is moderate, so the rating boost is intentionally small to avoid hiding textually relevant results.
- Using title + processed abstract improves search quality because some movies are found through plot words even when the title does not contain the query.

Limitations

- Some records have a rating of zero, which usually means no rating was available rather than very poor quality.
- The synonym mapping is useful but manually built, so it can be expanded with WordNet or embedding-based expansion later.
- The network endpoint summarizes relationships instead of rendering a full graph in the browser to keep the application lightweight.

Future Work

- Add genre extraction from abstracts and external metadata to support richer filtering.
- Use BERT or sentence embeddings for semantic search in addition to TF-IDF.
- Add Precision@K, Recall@K, MAP, and nDCG if a labeled relevance file becomes available.
- Render an interactive NetworkX graph in the interface for movie-term and movie-type relationships.

10. Conclusion

The final Movies Search Engine satisfies the main data mining and information retrieval requirements. It uses real collected data, applies preprocessing, builds searchable representations, processes user queries, ranks results with TF-IDF, improves matching through expansion and feedback, and presents everything through an advanced web interface. The bonus application turns the notebooks into a complete runnable project suitable for demonstration and discussion.